OpenMP — Lesson 7

Data in and out of a region: `firstprivate`, `lastprivate`, `default(none)`

Gustavo Lozano

November 2026

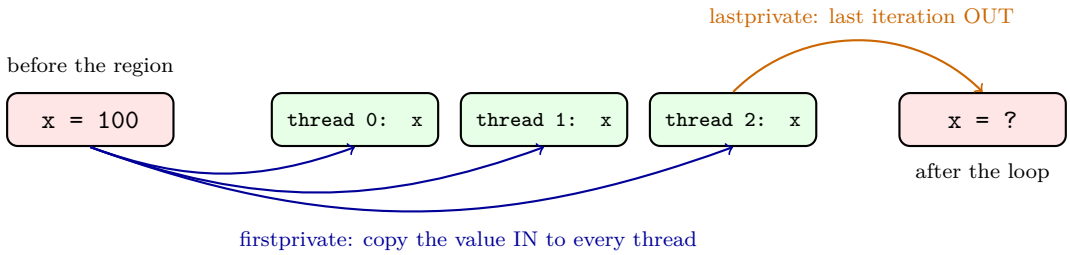
Source: Chapman, Jost & van der Pas [1], Ch. 4 "OpenMP Language Features" (data-sharing attribute clauses).

1 The idea

In Lesson 2 we saw that `private(x)` gives each thread its own copy of `x`. But that copy is cut off from the original in both directions:

- **in**: the copy starts *uninitialised*, it does not receive the value `x` had before the region;
- **out**: the copy is *discarded* at the end, nothing goes back to `x`.

Two clauses open one of these directions each, and a third one protects you from forgetting to decide.



2 The three clauses

Clause	Meaning
`private(x)`	private copy, **uninitialised**; discarded at the end (Lesson 2)
`firstprivate(x)`	private copy, **initialised** with the value of `x` before the region
`lastprivate(x)`	private copy; after a `for` (or `sections`), `x` receives the value from the **sequentially last** iteration (`i = n-1`), whichever thread ran it
`default(none)`	no implicit rules: every variable used in the region and declared outside it must appear in a clause, or the program **does not compile**

Why `default(none)` matters. Most OpenMP bugs are a variable that is shared by default when it should be private (Lesson 2). With `default(none)` you must write down a decision for every variable, and the compiler checks that you did. Using it in every parallel region is a widely recommended habit.

3 The code

data_clauses.c

```c
/*
 * data_clauses.c -- OpenMP Lesson 7: firstprivate, lastprivate, default(none).
 * Source: Chapman, Jost & van der Pas (2007), "Using OpenMP",
 *         Ch. 4 (data-sharing attribute clauses).
 *
 * Compile:
 *     gcc -Wall -fopenmp -o data_clauses data_clauses.c
 * Run:
 *     OMP_NUM_THREADS=4 ./data_clauses
 */

#include <stdio.h>
#include <omp.h>

#define N 8

int main(void)
{
    int offset = 100;   /* a value every thread needs as a STARTING point */
    int last   = -1;    /* we want the value from the LAST iteration      */
    int a[N];
    int n = N;

    /* ======== Part 1: firstprivate ========
       private(offset)      -> each thread gets a copy, UNINITIALISED.
       firstprivate(offset) -> each thread gets a copy, INITIALISED with
                               the value offset had before the region (100).
       default(none)        -> the compiler refuses any variable whose
                               sharing we did not state explicitly.        */
    printf("--- Part 1: firstprivate ---\n");
    #pragma omp parallel default(none) firstprivate(offset)
    {
        int id = omp_get_thread_num();   /* declared inside: private */
        offset += id;                    /* changes only THIS thread's copy */
        printf("  thread %d: offset = %d\n", id, offset);
    }
    /* The private copies are thrown away at the end of the region. */
    printf("  after the region: offset = %d\n", offset);

    /* ======== Part 2: lastprivate ========
       lastprivate(last) -> private inside the loop, and after the loop the
       original variable receives the value from the SEQUENTIALLY LAST
       iteration (i = n-1), no matter which thread executed it.           */
    printf("--- Part 2: lastprivate ---\n");
    #pragma omp parallel for default(none) shared(a, n) lastprivate(last)
    for (int i = 0; i < n; i++) {
        last = i * i;
        a[i] = last;
    }
    printf("  after the loop: last = %d   (a[n-1] = %d)\n", last, a[n - 1]);

    return 0;
}
```

```
gcc -Wall -fopenmp -o data_clauses data_clauses.c
OMP_NUM_THREADS=4 ./data_clauses
```

Typical output (line order in Part 1 will differ):

```
--- Part 1: firstprivate ---
```

```
  thread 0: offset = 100
  thread 1: offset = 101
  thread 2: offset = 102
  thread 3: offset = 103
  after the region: offset = 100
--- Part 2: lastprivate ---
  after the loop: last = 49   (a[n-1] = 49)
```

Key ideas

- Every thread starts from 100 because of `firstprivate`, and each changes only its own copy.
- After the region `offset` is still 100: private copies never flow back.
- `last = 49 = 7*7`: the value from `i = n-1`, the last iteration in *sequential* order, not from the thread that finished last.
- `a` and `n` are listed as `shared`: `n` is only read, and each thread writes different elements of `a`, so there is no race.

4 The experiment

Experiment

Question: what changes when each clause is removed or replaced?
Make one change at a time, recompile with `-Wall`, run, then undo it.

1. In Part 1 replace `firstprivate(offset)` by `private(offset)`. Read the compiler warning, then run.
2. In Part 2 replace `lastprivate(last)` by `private(last)`. Run.
3. In Part 2 replace `shared(a, n)` by `shared(a)`. Compile.

What to observe and answer.

1. With `private`, what values do the threads print? Why does the compiler warn?
2. Without `lastprivate`, what is `last` after the loop? Why?
3. What does the compiler say in step 3? What would have happened without `default(none)`?

Answers (read after trying). (1) Garbage, for example `32740, 32741, ...`: each thread adds its id to an uninitialised copy. GCC warns that `offset` is used uninitialised. The value before the region never reaches the threads. (2) `last = -1`, its value before the loop. The private copies held 49 and other values, but with `private` they are discarded and nothing is copied out. (3) `error: 'n' not specified in enclosing 'parallel'`. Without `default(none)`, `n` would silently be shared, which here happens to be correct. The point is that the compiler forces *you* to decide; in Lesson 2 the silent default was exactly what caused the race.

Next lesson. False sharing: why threads that write to *different* variables can still slow each other down (Chapman et al., Ch. 5).

References

[1] B. Chapman, G. Jost, and R. van der Pas. *Using OpenMP: Portable Shared Memory Parallel Programming*. MIT Press, Cambridge, MA, 2007.